UART SWITCH: MASTER + SLAVE

Pure bare-metal C on two NUCLEO-F446RE boards (USART1, 9600 baud, 8N1)

Part 1: the story | Part 2: night-before-exam deep dive

What the program does, in one breath

Press the USER button (PC13) on the **MASTER** board. It transmits the byte `'S'` over USART1. The **SLAVE** board receives it, blinks its LED 5 times, then transmits the byte `'D'` back. The master, which has been polling its receiver with a 6-second timeout, sees `'D'` and blinks twice to confirm. If nothing comes back it simply gives up.

Part	What you get	How to use it
1. The Story	The whole program as a short tale, each scene tagged to the code it represents	Read once, slowly. Builds the mental movie.
2A. Big picture	Crossed wiring, UART frame diagram, theory, the SPI / I2C / UART showdown	Skim first on exam night.
2B. Bit-level decode	Every register line in bits and hex, baud-rate math, flag table	Memorize the numbers.
2C. Code walkthrough	Master and slave loops, timeline, timing numbers	Be able to redraw the frame and timeline.
2D. Weak points + debug	Design flaws, improvements, troubleshooting table	Great for 'suggest an improvement' questions.
2E. Q and A + cram card	20 likely exam questions with model answers, one-page cheat sheet	Last 30 minutes before sleep.

> **How to read the callouts**
>
> **EXAM TIP** = something examiners love to ask. **TRAP** = a subtle behaviour that causes real bugs.

PART 1 - THE STORY

The Captain, the Guard, and the Two Walkie-Talkies

Two castles stand on distant hills. This time there is **no shared drum** and no party line. Each castle owns a walkie-talkie with *two separate wires*: a mouth wire (TX) and an ear wire (RX). The Captain's mouth is connected to the Guard's ear, and the Guard's mouth to the Captain's ear. Because each side has its own mouth and its own ear, either one may speak at any moment, even both at once. Here is what happens when the Captain presses his button.

Character	Real thing	Role in the story
Captain Master	The MASTER board	Owns the button (PC13); speaks first in this demo
Sleepy Guard	The SLAVE board	Owns the lantern (LED PA5); listens, acts, then answers
Mouth wire (TX)	PA9 on each board	What a castle says
Ear wire (RX)	PA10 on each board	What a castle hears. Pulled up so silence reads as HIGH
The shared floor	GND	Common 0 V reference
The wristwatch	Baud rate 9600 (BRR = 0x683)	Each castle's OWN clock, set to the same speed in advance
The word format	8N1 frame	Start knock, 8 letters (LSB first), stop silence
Secret words	`'S'` = 0x53, `'D'` = 0x44	Request word and Done word

Chapter 1 - Agreeing on the language

There is no beat wire, so the castles cannot keep time together. Instead, before the story begins, both Captain and Guard set their wristwatches to exactly the same speed: 9600 beats per second. They also agree on how every word is wrapped: the line rests HIGH when nothing is said; each word starts with one **start knock** (line pulled LOW for one beat), then eight bits, least significant first, then one **stop silence** (line HIGH). The knock lets the listener start his own watch at exactly the right moment, every single word. The mouth wires are connected crosswise to the ear wires. The ear wires also get a gentle pull-up rope, so an unplugged ear hears silence, not crackle. Power is switched on for the buildings and the walkie-talkie, and both mouth and ear are opened.

[code: RCC enables, GPIOA MODER/AFRH = AF7, PUPDR, USART1_BRR = 0x0683, CR1 = UE | TE | RE]

Chapter 2 - The button press

The Captain glances at his button every five milliseconds. On a fresh press he debounces for 20 ms, then does something sensible: he **empties his ear** of any stale old word, so he cannot mistake yesterday's reply for today's. He waits until his mouth is free (TXE) and speaks `'S'`.

[code: edge detect, delay_ms(20), read SR + DR, wait TXE, USART1_DR = 'S']

Chapter 3 - The Guard answers

The Guard's ear fills (RXNE) and he reads `'S'`. He lights his lantern five times (two seconds). Then, with no one asking, he simply opens his mouth and says `'D'`. No dummy messages, no envelopes, no house numbers. He talks because he has something to say, and his wire is his own.

[code: if (b == 'S') { blink(5); wait TXE; USART1_DR = 'D'; }]

Chapter 4 - The patient listener

Meanwhile the Captain does not guess how long the Guard will take. He checks his ear every millisecond, up to six thousand times. The instant a word arrives he reads it. If it is `'D'` he flashes his own lantern twice: mission confirmed. If six seconds pass in silence, he shrugs and goes home, so a missing Guard

can never trap him.

[code: t = 6000; while (t) { if RXNE: read DR, got = (DR == 'D'); break; delay_ms(1); t--; }]

Chapter 5 - Back to sleep

The Guard returns to listening. Any word that is not `'S'` is simply ignored. Both castles go dark.

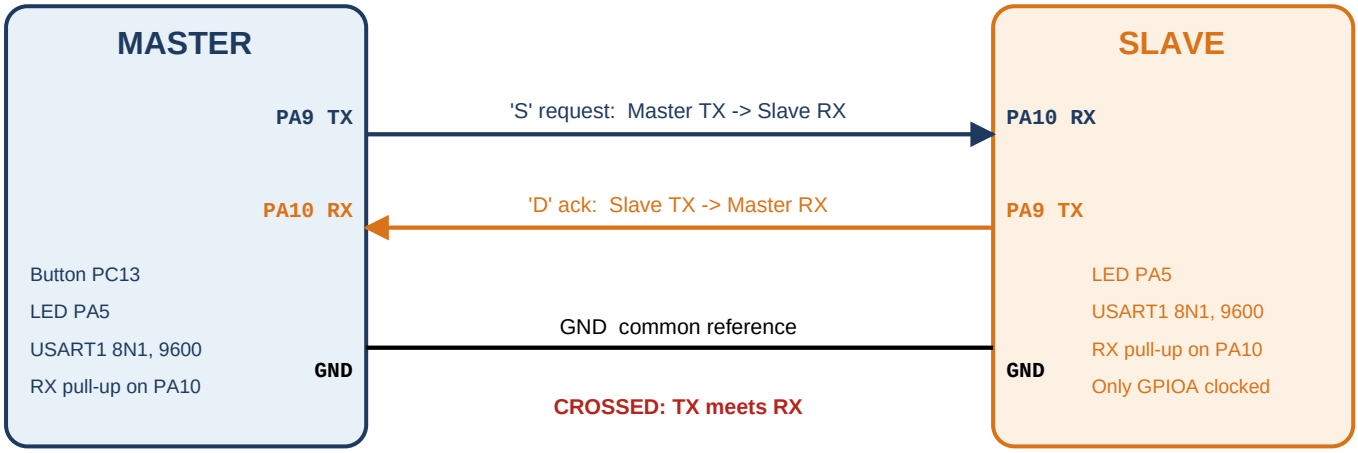
> **THE MORAL - three laws of UART hidden in this story**
>
> 1) **No shared clock**: both sides agree on speed and frame in advance, and each start bit re-synchronizes the receiver.
> 2) **Two independent lines**: either side may talk at any time (full duplex, peer to peer). The ack is just another message: no dummy byte, no preloaded reply, no address.
> 3) **It is point to point and blind to errors**: nothing in the frame says 'received OK'. The program has to invent its own acknowledge ('D') and its own timeout.

PART 2 - NIGHT-BEFORE-EXAM DEEP DIVE

2A. Big picture

60-SECOND SUMMARY

Two NUCLEO-F446RE boards on USART1: PA9 = TX, PA10 = RX (AF7), crossed (TX to RX), common GND. 9600 baud, 8 data bits, no parity, 1 stop bit (8N1), asynchronous, full duplex. BRR = 0x0683, CR1 = 0x200C (UE | TE | RE). Press -> master sends 'S' (0x53) -> slave receives, blinks x5 (2000 ms), sends 'D' (0x44) -> master, polling RXNE with a 6000 x 1 ms timeout, sees 'D' and blinks x2.



Unlike SPI and I2C (straight wiring), UART is **crossed**: TX on one side goes to RX on the other. Plugging TX to TX is the single most common UART wiring mistake.

Roles side by side

Item	MASTER project	SLAVE project
Clocks enabled	GPIOA, GPIOC (button), USART1	GPIOA, USART1
PA5	Output, LED (2 confirm blinks)	Output, LED (5 request blinks)
PA9 / PA10	AF7: USART1_TX / USART1_RX	Same
Pull resistors	PA10 (RX) pull-up, PC13 pull-up	PA10 (RX) pull-up
USART setup	BRR = 0x0683, CR1 = UE|TE|RE	Identical
Main loop	Polls button every 5 ms; sends 'S', polls RXNE up to about 6 s	Blocks on RXNE; on 'S' blinks and sends 'D'
Who talks first?	Master, by program design	Slave, but only after hearing 'S'

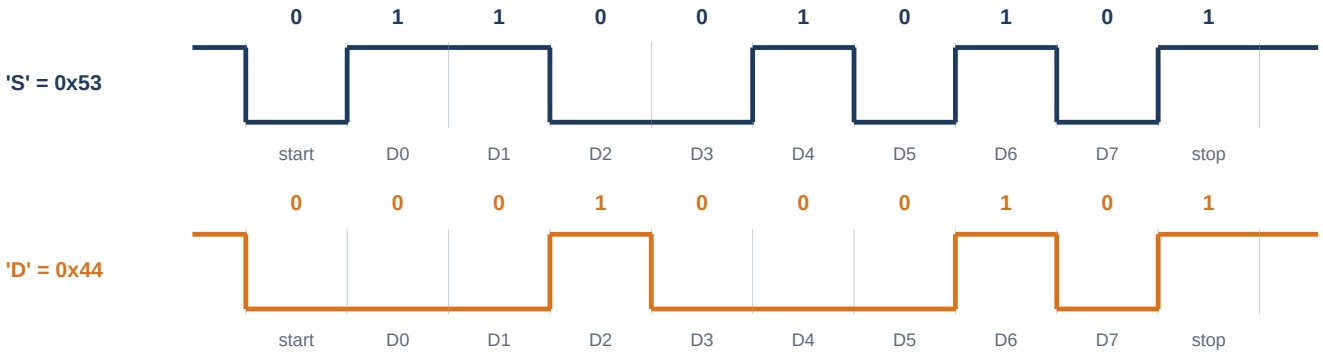
Note: UART has **no master or slave in the protocol**. Both ends are peers with identical USART setup. 'Master' and 'slave' are only roles chosen by the application.

UART theory you must be able to write from memory

- **Asynchronous:** no clock wire. Each side runs its own baud-rate clock set to the same value. Each frame's start bit re-synchronizes the receiver.
- **Frame (8N1):** idle HIGH; 1 start bit (LOW); 8 data bits, **LSB first**; no parity; 1 stop bit (HIGH). 10 bits per byte.
- **Baud rate:** bits per second. At 9600 baud: 1 bit = 104.17 us, one 10-bit frame = 1.04 ms, about 960 bytes per second.
- **Full duplex, point to point:** separate TX and RX lines; two devices per link (more needs RS-485 or extra lines).
- **Receiver oversampling:** the receiver samples each bit several times (16x here) and uses the middle samples with a vote, so small noise or clock drift does not corrupt data. A noisy bit sets the NF flag.

- **Errors you can detect:** framing error FE (stop bit not HIGH, usually wrong baud), noise NF, overrun ORE (new byte arrived before the old one was read), parity PE (if parity is enabled).
- **Tolerance:** both sides' baud clocks must match within a few percent over a frame; the HSI oscillator used here is about 1% accurate, which is fine at 9600 baud.

What 'S' and 'D' look like on the wire



Idle = HIGH. Data bits go LSB FIRST. 10 bits per byte; at 9600 baud 1 bit = 104.17 us, one frame = 1.04 ms.

The protocol showdown: same demo, three protocols

Aspect	SPI demo	I2C demo	UART demo
Wires	3 signals + GND (straight)	2 signals + GND + pull-ups (straight)	2 signals + GND (CROSSED)
Clock	Master drives SCK	Master drives SCL (slave may stretch)	None; both use baud 9600
Who may start	Master only	Master only	Either side
How the slave replies	Preloaded in DR, sent during the NEXT transfer	On demand during a READ transaction	Slave just transmits 'D' when it chooses
Dummy byte for the ack?	YES (0x00)	No (READ transaction)	No
How master waits	Blind 2500 ms delay	Blind 2500 ms delay	Polls RXNE, timeout about 6 s
Hang protection	None (infinite while loops)	Guard counters (GUARD = 200000)	Software timeout (6000 x 1 ms)
Acknowledge by hardware	None	ACK/NACK bit after each byte	None (the 'D' byte is the ack)
Duplex	Full	Half	Full
Typical devices per link	Many (CS per slave)	Many (addresses)	2

EXAM TIP - the UART demo is the only one with a real 'wait for reply'

SPI and I2C masters wait a fixed 2500 ms and then fetch the reply. The UART master waits for the reply to actually arrive (RXNE) and has a timeout. That is both simpler and faster, and the main reason UART is natural for 'command then answer' links.

2B. Bit-level decode of every register line

Memory map (base + offset = address)

Macro in code	Address	Base + offset	Notes
RCC_AHB1ENR	0x40023830	RCC 0x40023800 + 0x30	GPIO clocks
RCC_APB2ENR	0x40023844	RCC + 0x44	bit 4 = USART1EN (USART1 is on APB2)
GPIOA_MODER	0x40020000	GPIOA 0x40020000 + 0x00	pin modes
GPIOA_PUPDR	0x4002000C	GPIOA + 0x0C	pulls
GPIOA_ODR	0x40020014	GPIOA + 0x14	LED PA5
GPIOA_AFRH	0x40020024	GPIOA + 0x24	AF number for pins 8-15
GPIOC_PUPDR / IDR	0x4002080C / 0x40020810	GPIOC 0x40020800 + 0x0C / + 0x10	button PC13 (master)
USART1_SR	0x40011000	USART1 0x40011000 + 0x00	status
USART1_DR	0x40011004	USART1 + 0x04	data (write = TDR, read = RDR)
USART1_BRR	0x40011008	USART1 + 0x08	baud rate divider
USART1_CR1	0x4001100C	USART1 + 0x0C	control 1

Clock enables

Line	Bits set	Meaning
`RCC_AHB1ENR |= bits 0,2`	bit0, bit2	GPIOAEN, GPIOCEN (master). Slave sets only bit 0.
`RCC_APB2ENR |= (1<<4)`	bit4	USART1EN. Note APB2, not APB1 as for SPI2 and I2C1.

GPIO configuration with mask math

Pin n: `MODER` at bits [2n+1:2n]; `AFRH` for pins 8-15 at [4(n-8)+3 : 4(n-8)]. PA9 -> MODER 19:18 and AFRH 7:4; PA10 -> MODER 21:20, AFRH 11:8, PUPDR 21:20.

Register	Clear mask	Set value	Result
GPIOA_MODER (PA5)	3<<10	1<<10 = 0x400	PA5 output (LED)
GPIOA_MODER (PA9, PA10)	(3<<18)|(3<<20) = 0x003C0000	(2<<18)|(2<<20) = 0x00280000	bits 21:18 = 10 10, both pins alternate function
GPIOA_AFRH	(0xF<<4)|(0xF<<8) = 0x00000FF0	(7<<4)|(7<<8) = 0x00000770	PA9, PA10 = AF7 = USART1_TX, USART1_RX
GPIOA_PUPDR (PA10)	3<<20 = bits 21:20	1<<20 = 0x00100000	RX pull-up: line idles HIGH
GPIOC_PUPDR (PC13)	3<<26	1<<26 = 0x04000000	button pull-up (master only)

EXAM TIP - why a pull-up on RX?

A UART line idles HIGH. If the RX pin floats (wire unplugged or other board unpowered), noise can look like a start bit and produce garbage bytes. The pull-up keeps the idle level HIGH.

USART1 configuration

Register	Value	Meaning / calculation
USART1_BRR	0x0683 = 1667	USARTDIV = fCK / (16 x baud) = 16 MHz / (16 x 9600) = 104.1667. Mantissa = 104 = 0x68 (bits 15:4). Fraction = 0.1667 x 16 = 2.67, rounded to 3 (bits 3:0). BRR = (104 << 4) | 3 = 0x0683
Actual baud rate	about 9598	16 MHz / (16 x 104.1875) = 16 MHz / 1667 = 9598 baud. Error about -0.02%, negligible
USART1_CR1	0x200C	(1<<13) UE + (1<<3) TE + (1<<2) RE = 0x2000 + 0x8 + 0x4

USART1_CR1 bits

Bit	Name	Value here	Meaning
13	UE	**1**	USART enable
12	M	0	0 = 8 data bits
10	PCE	0	0 = no parity
3	TE	**1**	Transmitter enable
2	RE	**1**	Receiver enable
15	OVER8	0	0 = 16x oversampling
CR2[13:12]	STOP	00	00 = 1 stop bit (reset default)

This gives the classic **8N1** frame (8 data bits, No parity, 1 stop bit) without writing CR2 at all, because the reset defaults are already right.

USART1_SR flags

Bit	Flag	Meaning	Cleared by	Used in this code
7	**TXE**	Transmit data register empty: safe to write DR	Writing DR	before sending 'S' and 'D'
6	TC	Transmission complete (last frame fully shifted out)	Read SR then write DR	not used
5	**RXNE**	Received byte waiting in DR	Reading DR	slave wait; master ack poll
3	ORE	Overrun: new byte arrived while RXNE still set	Read SR, then read DR	stale-RX clearing
2	NF	Noise detected	Read SR then DR	not checked
1	FE	Framing error (stop bit was 0)	Read SR then DR	not checked
0	PE	Parity error	Read SR then DR	parity off

Mnemonic: **Write when TXE, Read when RXNE** (same as SPI). The one extra gotcha: **TXE says the buffer is free, not that the byte has left the wire**; TC says the byte has left.

Pin cheat table

Pin	Job	MODER bits	AFRH bits	Other
PA5	LED output	11:10 = 01	-	ODR bit 5
PA9	USART1_TX	19:18 = 10	7:4 = 0111 (AF7)	-
PA10	USART1_RX	21:20 = 10	11:8 = 0111 (AF7)	PUPDR 21:20 = 01 (pull-up)
PC13	USER button (active low)	input	-	PUPDR 27:26 = 01, IDR bit 13

USART2 (PA2/PA3) is normally wired to the ST-LINK virtual COM port on NUCLEO boards, so USART1 on PA9/PA10 is a clean choice for board-to-board links.

2C. Code walkthrough

Boilerplate both files share

```
void SystemInit(void) __attribute__((weak));        void SystemInit(void) { }
void __libc_init_array(void) __attribute__((weak)); void __libc_init_array(void) { }
```

Empty weak stubs satisfy the startup file's calls. SystemInit being empty leaves the clock at reset defaults (HSI 16 MHz), which BRR = 0x0683 and the delay loops assume. If the clock changed, the baud rate would be wrong and the link would break.

Master helpers

Function	Meaning
`pressed()`	Button is active-low: PC13 reads 0 when pressed
`blink_little()`	2 x (120 ms ON + 120 ms OFF) = 480 ms confirm
`delay_ms(ms)`	Software busy-wait, about 1 ms per outer iteration at 16 MHz

Master main loop, annotated

```
int prev = 0;
while (1) {
    int now = pressed();
    if (now && !prev) {                         // NEW press only
        delay_ms(20);                           // debounce
        if (pressed()) {
            (void)USART1_SR; (void)USART1_DR;   // clear stale RX (read SR then DR clears ORE/RXNE)
            while (!(USART1_SR & (1 << 7)));    // wait TXE
            USART1_DR = 'S';                    // transmit request

            unsigned int t = 6000; int got = 0; // timeout budget: 6000 x about 1 ms
            while (t) {
                if (USART1_SR & (1 << 5)) {     // RXNE: something arrived
                    if ((char)USART1_DR == 'D') got = 1;
                    break;                      // one byte only; stop either way
                }
                delay_ms(1); t--;
            }
            if (got) blink_little();            // confirmed
        }
    }
    prev = now;
    delay_ms(5);
}
```

EXAM TIP - why clear the stale RX first?

If an old byte (or a late 'D' from an earlier timeout) were still sitting in DR, the master's poll would read it instantly and wrongly accept it. Reading SR then DR empties the receiver and also clears any ORE.

Slave main loop, annotated

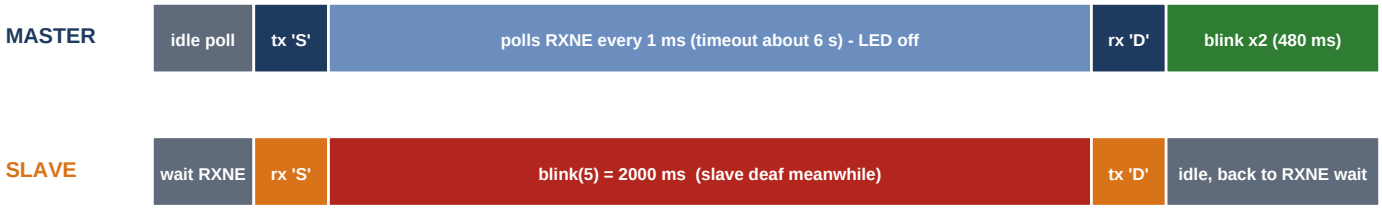
```
while (1) {
    while (!(USART1_SR & (1 << 5)));        // wait RXNE: a byte arrived
    char b = (char)USART1_DR;               // read it (clears RXNE)

    if (b == 'S') {
        blink(5);                           // 5 x (200 ON + 200 OFF) = 2000 ms
        while (!(USART1_SR & (1 << 7)));    // wait TXE
        USART1_DR = 'D';                    // transmit the ack, no one has to ask
    }                                       // any other byte is silently ignored
}
```

TRAP - the slave is deaf while blinking

During blink(5) nobody reads USART1_DR. If the master sent a second byte, it would sit in DR (RXNE set) and a third would set ORE and be lost. In this demo the master sends only one byte per press, so it works.

Full timeline of one button press



(segment widths NOT to scale; the master reacts the moment 'D' arrives)

#	When	Master	Wires	Slave
0	Power-up	Init, polls button every 5 ms	Both lines HIGH (idle)	Init, waits for RXNE
1	Press	Edge seen, 20 ms debounce, re-check; clears stale RX	idle	waiting
2	t0	Waits TXE, writes DR = 'S'	Master TX: start, 1 1 0 0 1 0 1 0, stop (1.04 ms)	RXNE = 1, reads 'S'
3	t0 .. t0+2000 ms	Polls RXNE every 1 ms	idle	blink(5)
4	t0+2000 ms	still polling	Slave TX: 'D' frame (1.04 ms)	waits TXE, writes DR = 'D'
5	t0+2001 ms	RXNE = 1, reads 'D', got = 1	idle	back to waiting
6	after	blink_little() (480 ms)	idle	idle

Timing numbers to memorize

Quantity	Value	How
Baud rate	9600 (actual 9598)	16 MHz / 1667
One bit / one frame	104.17 us / 1.04 ms	1 / 9600; times 10 (start + 8 data + stop)
Slave blink time	2000 ms	5 x (200 + 200), BLINK_MS = 200
Master timeout	about 6 s	6000 loops x delay_ms(1), plus small overhead
Margin	about 4 s	6000 ms - 2000 ms
Master success blink	480 ms	2 x (120 + 120)
Debounce / poll	20 ms / 5 ms	delay_ms values

2D. Weak points, improvements, debugging

Design weaknesses (great for 'how would you improve this?')

Weakness	Why it matters	Better design
Error flags never checked	A framing, noise or overrun error silently corrupts or drops a byte; nobody knows.	Check FE / NF / ORE in SR; discard and retry.
No retry, no negative reply	Slave ignores anything but 'S'; the master cannot tell 'slave dead' from 'slave misheard' (both = timeout).	Add an error byte and a retry count.
Master accepts only the first byte	Any byte other than 'D' (noise, stale data) ends the wait as failure.	Keep polling until 'D' or timeout; ignore unexpected bytes.
Slave deaf during blink()	A second command during 2 s would overrun and be lost.	Use the RXNE interrupt or ring buffer; non-blocking blink.
Polling and software delays	CPU is busy; the timeout depends on loop speed and optimization.	SysTick or a timer for the timeout, interrupts or DMA for RX.
Uncalibrated clock	Both boards run from the HSI RC oscillator (about 1%); fine at 9600 but risky at high baud.	Use a crystal or the ST-LINK clock, or a lower-error baud setting.
No protocol structure	Single-byte commands: no length, address or checksum.	Frame with header, length and CRC; use RS-485 for many devices.

Troubleshooting table

Symptom	Likely cause	Check
Nothing happens at all	TX wired to TX (not crossed); no common GND; USART1 clock or CR1 enables missing	Master PA9 to slave PA10, master PA10 to slave PA9, GND joined; CR1 = 0x200C
Slave blinks randomly	Floating RX (no pull-up, wire unplugged), noise, no ground	PA10 pull-up set; short wires; shared GND
Garbage or wrong letters	Baud mismatch; one board on a different clock; FE set	Both BRR = 0x0683; both at HSI 16 MHz
Slave blinks, master never confirms	Slave TX (PA9) to master RX (PA10) wire missing; reply arrived after the 6 s timeout	Check the second wire; BLINK_MS x 10 must stay below 6000
Master confirms instantly with no slave	Stale RX not cleared (older code); wire loopback	Clear SR/DR before sending; no TX-to-RX shorting on the same board
Pins do nothing	AFRH not AF7, wrong MODER, or GPIOA clock off	AFRH = 0x770 on bits 11:4; MODER bits 21:18 = 1010

2E. Likely exam questions with model answers

Q1. Why is UART called asynchronous?

A. There is no clock wire. Each end uses its own baud-rate clock set to the same value in advance, and each frame's start bit re-synchronizes the receiver.

Q2. Draw the frame for 'S' (0x53) at 8N1.

A. Idle HIGH, start bit 0, then data bits LSB first: 1 1 0 0 1 0 1 0, then stop bit 1. That is 10 bits.

Q3. Why must TX be connected to RX?

A. The transmitter of one device must feed the receiver of the other. TX-to-TX leaves both listening to nothing and driving against each other.

Q4. How is 9600 baud obtained from BRR = 0x0683?

A. USARTDIV = 16 MHz / (16 x 9600) = 104.1667. Mantissa 104 = 0x68, fraction 0.1667 x 16 = 2.67 rounds to 3. BRR = (104 << 4) | 3 = 0x683.

Q5. What is the actual baud rate and error?

A. 16 MHz / 1667 = 9598 baud, about -0.02% error, far below the few-percent tolerance.

Q6. What happens if the two sides use different baud rates?

A. The receiver samples at the wrong instants, giving garbage bytes and framing errors (FE), especially noticeable after the first few bits of a frame.

Q7. Why PA9 / PA10 and AF7?

A. The datasheet alternate-function table maps USART1_TX and USART1_RX to PA9 and PA10 as AF7. AFRH holds the 4-bit AF number for pins 8-15 (0x770 for these two pins).

Q8. Why enable a pull-up on the RX pin?

A. UART idles HIGH. A floating RX can show noise that looks like start bits; the pull-up keeps it HIGH.

Q9. Decode CR1 = 0x200C.

A. UE (bit 13) = USART enable, TE (bit 3) = transmitter enable, RE (bit 2) = receiver enable. M = 0 (8 bits), PCE = 0 (no parity).

Q10. TXE vs TC vs RXNE?

A. TXE: transmit data register empty, safe to write the next byte. TC: the last frame has completely left the shifter. RXNE: a received byte is waiting; reading DR clears it.

Q11. How are RXNE, TXE and ORE cleared?

A. RXNE: read DR. TXE: write DR. ORE: read SR, then read DR.

Q12. Why does the master clear stale RX before sending?

A. So an old byte or late reply cannot be mistaken for the new ack. Reading SR then DR empties the receiver and clears ORE.

Q13. How does the master avoid hanging if the slave is missing?

A. It polls RXNE inside a loop of at most 6000 iterations with delay_ms(1), about 6 s, and continues without confirmation on timeout.

Q14. Why does UART need no dummy byte or preload, unlike SPI?

A. UART has separate TX and RX lines and no master clock, so the slave can transmit whenever it wants. The ack is simply another message.

Q15. Is UART master-slave?

A. No. It is peer to peer; both ends can transmit at any time. The master and slave roles here are only application choices.

Q16. What would happen if the slave blinked for 7 s?

A. The master's 6 s timeout would expire and it would give up with no confirmation. The slave's 'D' arrives later and sits in the master's RX register; the next button press clears it as stale data before sending.

Q17. What are parity and stop bits, and are they used here?

A. Parity is an optional extra bit for single-bit error detection (PCE = 1). Stop bits (1, 1.5 or 2) give the receiver recovery time. Here: no parity, 1 stop bit.

Q18. Compare how the acknowledge works in SPI, I2C and UART.

A. SPI: slave preloads the reply, master sends a dummy byte, reply arrives one transfer late. I2C: hardware ACK/NACK bit per byte, and the slave replies during the READ transaction. UART: no hardware ack; the slave sends an ordinary byte 'D' and the master times out.

Q19. What problems would arise if the master sent 'S' twice in a row while the slave blinks?

A. The first extra byte waits in the slave's DR; a second would overrun (ORE) and be lost. The slave is deaf during blink(5) because it polls instead of using interrupts.

Q20. Name two improvements.

A. (1) Check FE/NF/ORE and add retries with an error reply. (2) Use an RXNE interrupt and a timer for the timeout instead of blocking loops.

ONE-PAGE CRAM CARD

Read this last, then sleep

Topic	Remember
Flow	Press -> master TX 'S' (0x53) -> slave RX, blinks x5 (2000 ms) -> slave TX 'D' (0x44) -> master polls RXNE (6000 x 1 ms) -> 'D' -> blinks x2
Wiring	CROSSED: master PA9 TX to slave PA10 RX; master PA10 RX to slave PA9 TX; GND. AF7. RX pull-up.
Frame	8N1: idle HIGH, start 0, 8 data bits LSB first, stop 1. 10 bits per byte.
Baud	BRR = 0x0683 = (104 << 4) | 3. USARTDIV = 16 MHz / (16 x 9600) = 104.1667. Actual 9598 baud. 1 bit = 104 us; frame = 1.04 ms.
CR1	0x200C = UE(13) + TE(3) + RE(2). M = 0 (8 bits), PCE = 0 (no parity). CR2 STOP = 00 (1 stop bit).
SR flags	7 TXE (write), 6 TC, 5 RXNE (read), 3 ORE, 2 NF, 1 FE, 0 PE
Clearing	RXNE: read DR. TXE: write DR. ORE/NF/FE/PE: read SR then DR. TC: read SR then write DR.
Masks	MODER PA9/PA10: 0x3C0000 -> 0x280000; AFRH 0xFF0 -> 0x770; PUPDR PA10 pull-up 1<<20
Clocks	AHB1ENR bit0 (GPIOA), bit2 (GPIOC, master); APB2ENR bit4 = USART1
Key traps	TX to TX is wrong. Clear stale RX before sending. TXE is not TC. Slave deaf during blink. Baud must match.
Weak points	No error-flag checks, no retry, polling, deaf slave, HSI accuracy, single-byte protocol
UART vs SPI vs I2C	UART: async, 2 crossed lines, peer to peer, 2 devices. SPI: sync, 4-wire, fastest, preload reply. I2C: sync, 2 shared wires, address + ACK.

7-STEP RECIPE FOR UART ON STM32 (REGISTER LEVEL)

1) Enable GPIO and USART clocks (USART1 on APB2). 2) Pins to AF mode. 3) Select AF7. 4) RX pull-up. 5) BRR = fCK / (16 x baud) as mantissa:fraction. 6) CR1 = UE | TE | RE. 7) Send: wait TXE then write DR. Receive: wait RXNE then read DR (with a timeout).

Mnemonics: **TX talks, RX listens**, and they must cross. **Idle is HIGH**, start is LOW. **LSB goes first**. **Write when TXE, read when RXNE.** No clock wire means the baud rate must match on both ends.